Using GitHub Copilot Chat in Visual Studio Code

Learning Objectives

By the end of this lesson, you should be able to:

- Explain the purpose of GitHub Copilot Chat.
- Access and use Copilot Chat inside Visual Studio Code.
- Ask theoretical and practical programming questions.
- Use follow-up questions to refine an ongoing conversation.
- Understand how the current development context can influence Copilot's responses.
- Recognize the difference between a vague prompt and a context-aware prompt.
- Ask Copilot to generate functions and code snippets.
- Copy generated code or insert it directly into an open source file.
- Create new chat sessions and work with previous conversations.
- Use Copilot Chat without assuming that its responses are automatically correct.

---

1. From Code Completion to Conversational Assistance

So far, we have examined:

- what GitHub Copilot is;
- how AI-assisted programming works;
- how development context affects suggestions;
- how developer intent influences results;
- how to craft clearer and more specific prompts.

We can now begin using Copilot more interactively.

One important method of interacting with GitHub Copilot is **GitHub Copilot Chat**.

Instead of waiting for Copilot to suggest code while you type, Copilot Chat allows you to actively communicate with the AI using natural-language prompts.

The basic interaction is:

```
Developer asks a question
          ↓
Copilot interprets the request
          ↓
Relevant context may be considered
```

```
          ↓
Copilot generates a response
          ↓
Developer reviews the response
          ↓
Continue / Refine / Use / Reject
```

This turns Copilot from a code-completion mechanism into a **conversational coding assistant**.

---

2. What Is GitHub Copilot Chat?

GitHub Copilot Chat provides a chat-based interface for interacting with GitHub Copilot directly from your development environment.

Instead of communicating only through partially written code, you can ask questions such as:

What is an API?

or:

What does HTTP status 500 mean?

You can also ask practical programming questions:

How do I add a CSS stylesheet to an HTML file?

or request code:

Create a JavaScript function that returns the month from a Date object.

Copilot Chat can therefore assist with several categories of development tasks.

Task	Example
Concept explanation	What is an API?
Protocol explanation	What is HTTP status 500?
Syntax assistance	How do I link a stylesheet to HTML?
Code generation	Create a JavaScript function…
Code explanation	Ask Copilot to explain existing code

Problem solving	Ask how to implement a particular operation
Iterative refinement	Ask follow-up questions about a previous response

NOTE — Conversational Coding Assistant

Copilot Chat allows you to describe programming problems using natural language rather than requiring you to know the exact syntax or terminology before asking for assistance.

---

3. Why Use Copilot Chat?

Developers frequently interrupt their coding workflow to search for information.

A traditional workflow might look like:

```
Encounter a question
        ↓
Leave the IDE
        ↓
Open a browser
        ↓
Search the web
        ↓
Read several results
        ↓
Find a relevant example
        ↓
Return to the IDE
        ↓
Continue coding
```

With an IDE-integrated assistant, some questions can instead be handled through:

```
Encounter a question
        ↓
Open Copilot Chat
        ↓
Ask the question
        ↓
Review the response
        ↓
Continue coding
```

This can reduce **context switching** and help developers remain focused on the current development task.

Copilot Chat can be particularly useful for:

- quick programming questions;
- syntax reminders;
- code examples;
- explanations;
- unfamiliar APIs;
- repetitive implementation tasks;
- code-generation assistance.

TIP — Use Chat to Reduce Small Interruptions

Copilot Chat is particularly convenient for questions that would otherwise require leaving the IDE for a short search. Keeping the interaction inside the development environment can help preserve your development flow.

---

4. Copilot Chat Is Still an AI System

The conversational interface can make Copilot's responses appear authoritative.

However, the same fundamental limitation still applies:

AI response ≠ guaranteed correct answer

Copilot may occasionally:

- misunderstand your question;
- make incorrect assumptions;
- generate invalid code;
- use an inappropriate API;
- overlook an edge case;
- provide outdated or unsuitable approaches;
- produce code that works but does not fit your architecture.

Therefore, the correct workflow remains:

Ask
↓
Read
↓
Understand
↓
Verify

↓

```
Use or modify
```

WARNING — Conversational Does Not Mean Authoritative

A detailed explanation or professionally formatted code sample can still be incorrect. Treat Copilot responses as assistance that requires developer judgment.

---

5. Opening Copilot Chat in Visual Studio Code

Let's now use Copilot Chat inside Visual Studio Code.

Step 1 — Start Visual Studio Code

Open Visual Studio Code.

Before continuing, confirm that GitHub Copilot has already been installed and configured.

You should have:

Visual Studio Code ✓

GitHub account signed in ✓

GitHub Copilot available ✓

Copilot ready ✓

Step 2 — Check Copilot Status

Locate the Copilot controls or icon in Visual Studio Code.

Check that Copilot indicates that it is available or ready.

If Copilot is not available, verify:

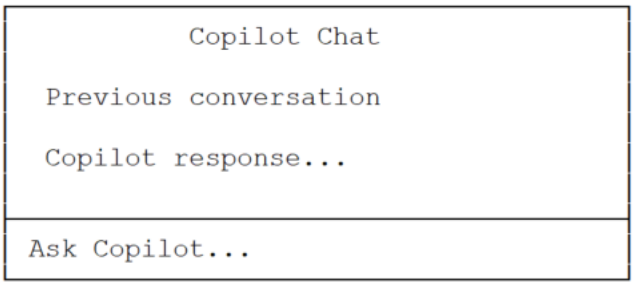
1. the extension or Copilot functionality is installed;
2. you are signed into GitHub;
3. the correct GitHub account is being used;
4. the account has Copilot access;
5. VS Code has Internet connectivity.

Step 3 — Open Copilot Chat

Use the Copilot Chat control available in your version of Visual Studio Code.

The exact interface can change between VS Code and Copilot versions, but you should be presented with a conversational interface containing an input area.

Conceptually:



NOTE — Interface Differences

Visual Studio Code and GitHub Copilot evolve frequently. Buttons, icons, panel locations, and menu names may differ from screenshots used in class. Focus on the functionality rather than memorizing a specific UI position.

---

6. Lab 1 — Ask a Theoretical Question

We will begin with a simple programming concept.

In Copilot Chat, enter:

What is an API?

Submit the prompt.

Copilot should generate an explanation.

What Just Happened?

The interaction was:

```
Natural-language question
          ↓
Copilot Chat
          ↓
AI processing
          ↓
Natural-language explanation
```

No code generation was required.

This demonstrates that Copilot Chat can be used for **conceptual programming assistance**, not just code completion.

---

7. Asking Independent Questions

You can ask another programming question in the same chat.

For example:

What is HTTP status 500?

Copilot should explain that HTTP status code 500 represents an internal server error and provide additional context.

This demonstrates that Copilot Chat can be used as an interactive programming reference.

Potential questions include:

- What is REST?
- What is dependency injection?
- What is the difference between GET and POST?
- What is asynchronous programming?
- What is an HTTP status code?

TIP — Use Copilot for Concepts, Not Only Code

When you encounter unfamiliar terminology while programming, asking for an explanation before requesting code can help you understand the solution rather than simply copying it.

---

8. Follow-Up Questions and Conversational Context

One important feature of chat-based interaction is the ability to ask **follow-up questions**.

Suppose you first ask:

What is an API?

You could then ask:

How is that different from a REST API?

The second question depends on the previous conversation.

You do not necessarily need to repeat the entire original question because the conversation provides additional context.

Conceptually:

```
Question 1
    ↓
Answer 1
    ↓
Follow-up Question
    ↓
Previous Conversation + New Question
    ↓
More Focused Answer
```

This is useful when progressively exploring a topic.

---

9. Practical Questions with Code Examples

Copilot Chat can also answer questions that combine explanation with code.

Consider:

How do I add a stylesheet to an HTML file?

A useful response may explain that an external stylesheet can be referenced from the <head> section of an HTML document using a <link> element.

For example:

```
<!DOCTYPE html>
<html>
<head>
    <link rel="stylesheet" href="style.css">
</head>
<body>

</body>
</html>
```

The response therefore contains two forms of assistance:

```
Explanation
     +
Code Example
```

This can be useful because the developer learns:

- what element to use;

- where it belongs;
- how it is structured;
- how the CSS file is referenced.

NOTE — Explanation + Example

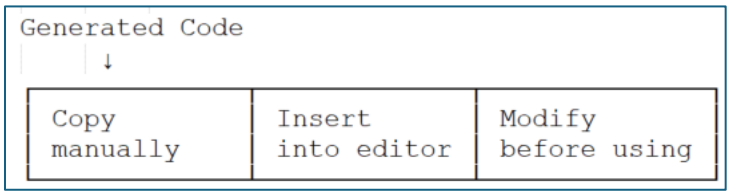
A useful Copilot response often combines conceptual explanation with implementation code. Read both. The explanation helps you understand *why* the code is structured the way it is.

---

10. Working with Generated Code

When Copilot Chat produces a code block, several workflows may be available.

You may:



The exact available actions depend on the current VS Code/Copilot interface.

---

11. Copying Generated Code

One option is to copy the generated code.

For example, Copilot may generate:

```
function getMonthFromDate(date) {
    return date.getMonth() + 1;
}
```

You can copy the code and paste it into the appropriate source file.

This gives you complete control over where the code is inserted.

---

12. Inserting Code into the Editor

Copilot may also provide an option to insert generated code directly into the active editor.

Suppose you have:

sample.js
open and your cursor is positioned where you want the new function.

You can use the available insert action to place the generated code into the editor.

The workflow becomes:

```
Ask Copilot
     ↓
Receive code
     ↓
Review code
     ↓
Insert at cursor
     ↓
Continue editing
```

WARNING — Review Before Inserting

Direct insertion is convenient, but convenience should not replace review. Read generated code before incorporating it into your project.

---

13. Creating a New Chat

At some point, you may want to begin a completely different conversation.

Copilot Chat provides a way to create a new chat session.

Conceptually:

```
Current Chat
    ↓
New Chat
    ↓
Fresh Conversation
```

Starting a new conversation is useful when:

- changing topics;
- starting an unrelated programming task;
- avoiding unnecessary conversational context;
- separating different problems.

For example, you might use one conversation for:

Understanding REST APIs

and another for:

Debugging a JavaScript function

Keeping unrelated tasks separate can make conversations easier to follow.

---

14. Chat History

Starting a new conversation does not necessarily mean previous conversations are permanently lost.

Copilot provides mechanisms for accessing previous chat sessions or conversation history, depending on the current product interface and configuration.

This allows you to return to earlier questions and responses.

NOTE — Chat History Behavior

History availability and retention behavior can depend on the current GitHub Copilot and VS Code implementation. Do not assume that every conversation will always be retained indefinitely.

---

15. Changing the Chat Layout

Copilot Chat does not necessarily have to remain in one fixed panel.

Depending on your VS Code version, you may be able to:

- keep Chat in a side panel;
- open Chat in an editor;
- open or move Chat into another window or layout.

This can be useful when working with a large amount of generated content.

For example:

Window 1

Source Code
JavaScript
HTML
CSS

and:

Window 2

Copilot Chat
Questions
Explanations
Generated Code

This arrangement can be convenient when comparing code with a longer AI response.

---

16. Context-Aware Chat

We now reach one of the most important concepts in this lesson.

Copilot Chat can use relevant development context to help interpret a question.

This means that the same natural-language question can potentially produce different responses depending on the surrounding context.

Let's demonstrate this.

---

17. Lab 2 — Asking a Vague Question Without Useful Context

Suppose we ask:

How to get a month from date?

Examine the prompt carefully.

It does **not** specify:

- JavaScript;
- Java;
- C#;
- Python;
- another language.

Therefore, the request is ambiguous.

Conceptually:

How to get a month from date?
↓
Which programming language?

↓

Unknown

Copilot must either:

- make an assumption;
- provide a generic answer;
- choose a likely language;
- ask for clarification.

For example, it might respond with a Python solution.

That does not necessarily mean Copilot failed.

The developer failed to provide an important piece of information.

PITFALL — Blaming the AI for Missing Context

If your prompt is ambiguous, multiple answers may be valid. Before deciding that Copilot produced the "wrong" answer, check whether your prompt clearly communicated the requirement.

---

18. Lab 3 — Provide Context Through the Development Environment

Now let's change the situation.

Step 1 — Open a Project Folder

Open a folder in Visual Studio Code.

It may be:

- a new folder;
- an existing project;
- a simple practice workspace.

Step 2 — Create a JavaScript File

Create:

`sample.js`

Open the file in the editor.

Now the active development environment clearly indicates that JavaScript is relevant to the current task.

Step 3 — Ask the Same Question

Ask exactly the same question:

How to get a month from date?

Notice that the natural-language prompt itself has not changed.

But the environment has changed.

Previously:

Question
+
Little relevant development context

Now:

Question
+
Open JavaScript file
+
Workspace context

Copilot may therefore interpret the question as a JavaScript-related request.

---

19. Understanding the Difference

The comparison is important.

Situation A

Prompt: How to get a month from date?

Development context: No clear programming-language signal

Possible result: Copilot assumes a language

Situation B

Prompt: How to get a month from date?

Development context: sample.js is active/relevant

Possible result: JavaScript-oriented answer

The prompt is identical.

The **context** is different.

Therefore:

```
Same Prompt
     +
Different Context
     ↓
Potentially Different Response
```

This demonstrates why understanding context is essential when working with Copilot.

---

20. References Used by Copilot

In some Copilot interactions, the interface may indicate that particular files or resources were used as references.

For example, you might see that:

`sample.js`

was considered relevant to the response.

This provides useful insight into why Copilot interpreted the question in a particular way.

Conceptually:

```
Prompt
   +
sample.js
   ↓
JavaScript context
   ↓
JavaScript-oriented response
```

TIP — Inspect Context Indicators

When the Copilot interface shows references or contextual sources, examine them. They can help you understand what information influenced the generated response.

---

21. Context Does Not Replace Good Prompting

The previous example demonstrates that Copilot can infer useful information from your environment.

However, this does **not** mean you should deliberately write vague prompts.

Compare:

How to get a month from date?

with:

Create a JavaScript function that accepts a Date object

and returns the month as a number.

The second prompt explicitly communicates:

Language → JavaScript
Task → Create function
Input → Date object
Output → Month number

The stronger approach is:

```
Useful Development Context
          +
Clear Developer Prompt
          ↓
Better-defined task
```

TIP — Context and Prompt Quality Work Together

Do not rely entirely on Copilot to infer what you mean. Relevant development context is valuable, but explicitly stating important requirements usually reduces ambiguity.

---

22. Lab 4 — Generate a Function with Copilot Chat

We will now ask Copilot to create code rather than merely explain syntax.

Assume `sample.js` is open.

Start with:

Create a function.

This prompt is technically valid but extremely vague.

Questions remain:

- What should the function do?
- What input should it receive?
- What should it return?
- What should the function be called?

Copilot can still generate something, but it must make several assumptions.

---

23. Improve the Prompt

Now combine the intent with the previous task:

Create a function to get the month from a date.

Because the current development context includes a JavaScript file, Copilot may produce something similar to:

```
function getMonthFromDate(date) {
    return date.getMonth() + 1;
}
```

The exact generated implementation may vary.

---

24. Understand the Generated Code

Never stop at code generation.

Examine the implementation:

```
function getMonthFromDate(date) {
    return date.getMonth() + 1;
}
```

Function Declaration

`function getMonthFromDate(date)`

The function:

- is named getMonthFromDate;
- accepts one parameter named date.

Extracting the Month

```
date.getMonth()
```

JavaScript's `Date.getMonth()` returns a zero-based month value.

Conceptually:

```
January   → 0
February  → 1
March     → 2
...
December  → 11
```

Therefore:

```
date.getMonth() + 1
```

converts it into the more familiar:

```
January   → 1
February  → 2
...
December  → 12
```

This is exactly the type of detail developers must understand before accepting generated code.

---

25. Use the Generated Function

Copilot may also demonstrate how the function can be called.

For example:

```
const today = new Date();
const month = getMonthFromDate(today);

console.log(month);
```

Let's examine the steps.

Step 1 — Create a Date

```
const today = new Date();
```

This creates a Date object representing the current date and time.

Step 2 — Call the Function

```
const month = getMonthFromDate(today);
```

The Date object is passed to the generated function.

Step 3 — Display the Result

```
console.log(month);
```

The resulting month number is printed to the console.

---

26. Insert the Function into the Source File

If Copilot provides an insert action:

1. Place the cursor at the desired location in sample.js.
2. Review the generated function.
3. Select the appropriate insert action.
4. Verify where the code was inserted.
5. Format or modify the code if necessary.

Your file might now contain:

```
function getMonthFromDate(date) {
    return date.getMonth() + 1;
}

const today = new Date();
const month = getMonthFromDate(today);

console.log(month);
```

---

27. Run and Verify the Code

Generating and inserting code is not the end of the development process.

You should run it.

For example, with an appropriate JavaScript runtime:

```
Run sample.js
      ↓
Observe output
      ↓
Compare output with expected result
```

If the current month is August, for example, the expected result would be:

8

Then test other dates.

```
const date1 = new Date(2026, 0, 15);
console.log(getMonthFromDate(date1));
```

Expected:

1

Try December:

```
const date2 = new Date(2026, 11, 15);
console.log(getMonthFromDate(date2));
```

Expected:

12

TIP — Turn Generated Examples into Tests

When Copilot generates a function, immediately try several inputs. Testing helps you understand both the generated implementation and the underlying API behavior.

---

28. The Complete Copilot Chat Workflow

The practical workflow covered in this lesson can be summarized as:

```
Open project
     ↓
```

```
Open relevant source file
      ↓
Open Copilot Chat
      ↓
Ask a programming question
      ↓
Copilot considers prompt + relevant context
      ↓
Review response
      ↓
Ask follow-up questions if necessary
      ↓
Generate code
      ↓
Review generated code
      ↓
Copy or insert code
      ↓
Run and test
      ↓
Modify if necessary
```

This is considerably different from:

```
Ask Copilot
      ↓
Insert code
      ↓
Done
```

The second workflow removes the most important component:

developer judgment.

---

29. Practical Exercise — Experiment with Context

Create two files:

```
sample.js
sample.py
```

Experiment A

Open **`sample.js`**.

Ask:

How do I get the current date?

Observe the answer.

Experiment B

Open `sample.py`.

Ask the same question:

How do I get the current date?

Observe whether the response changes.

Questions

1. Did Copilot recognize the active language?
2. Did the response reference the active file?
3. Did the generated syntax change?
4. Was the prompt itself sufficient to determine the language?
5. How did development context affect the result?

NOTE — Results Can Vary

AI responses are not necessarily deterministic. Different models, Copilot versions, workspace configurations, and context-selection behavior may produce different results. Focus on understanding the relationship between **prompt + context + generated response**.

---

30. Practical Exercise — Improve Prompt Quality

Start with:

Create a function.

Then progressively improve it.

Version 1

Create a function.

Version 2

Create a function that extracts the month from a date.

Version 3

Create a JavaScript function that extracts the month from a Date object.

Version 4

Create a JavaScript function named getMonthFromDate

that accepts a Date object and returns the month as an integer

from 1 to 12.

Compare the responses.

Identify how each version improves:

- context;
- intent;
- clarity;
- specificity.

The progression is:

```
Vague
  ↓
Intent added
  ↓
Language added
  ↓
Input specified
  ↓
Output specified
  ↓
More precise request
```

---

31. Common Pitfalls

Pitfall 1 — Asking Extremely Vague Questions

Example:

Fix this.

Without sufficient context, Copilot may not know:

- what is broken;
- what behavior is expected;

- what constraints exist.

Improve the request by explaining the problem.

Pitfall 2 — Assuming Copilot Knows the Desired Language

Example:

How do I sort a list?

This could refer to:

- Java;
- Python;
- JavaScript;
- C#;
- C++;
- many other languages.

Relevant project context may help, but explicitly specifying the language removes unnecessary ambiguity.

Pitfall 3 — Copying Code Without Reading It

Avoid:

```
Generate
   ↓
Copy
   ↓
Assume correct
```

Use:

```
Generate
   ↓
Read
   ↓
Understand
   ↓
Test
   ↓
Modify
```

Pitfall 4 — Assuming Context Is Always Perfect

Copilot may have access to relevant information and still misunderstand your intention.

If a response is inappropriate:

1. inspect the prompt;
2. inspect the available context;
3. clarify the language;
4. state the desired behavior;
5. specify inputs and outputs;
6. ask again.

---

32. Recommended Workflow for Copilot Chat

For intermediate and advanced developers, a disciplined workflow is:

Step 1 — Establish Context

Open or reference the files relevant to your current task.

Step 2 — State the Intent

Explain what you are trying to accomplish.

Step 3 — Add Important Requirements

Specify:

- inputs;
- outputs;
- libraries;
- frameworks;
- constraints;
- expected behavior.

Step 4 — Submit the Prompt

Allow Copilot to generate a response.

Step 5 — Read the Explanation

Do not immediately jump to the code.

Step 6 — Inspect the Code

Ask:

- Does it make sense?
- Does it match our architecture?
- Are the APIs appropriate?
- Are there edge cases?

Step 7 — Refine if Necessary

Ask a follow-up question.

Step 8 — Insert or Copy

Only after reviewing the result.

Step 9 — Run and Test

Verify actual behavior.

Step 10 — Modify

Treat generated code as a starting point when necessary.

---

33. Quick Reference — What Can You Ask Copilot Chat?

Goal	Example Prompt
Learn a concept	What is an API?
Understand HTTP	What does HTTP status 500 mean?
Learn syntax	How do I add a stylesheet to HTML?
Generate code	Create a JavaScript function…
Refine code	Modify this function so that…
Ask a follow-up	Can you explain why this works?
Request an example	Show me an example of using this function.
Clarify behavior	What happens if the input is null?

---

34. Key Concepts

Concept	Meaning
Copilot Chat	Conversational interface for interacting with GitHub Copilot
Prompt	Natural-language instruction or question
Follow-up Prompt	A new request that builds on the existing conversation
Development Context	Relevant information from the coding environment that can help interpret the task
Reference	File or contextual information used in producing a response
Generated Code	Code proposed by the AI
Insert	Place generated code into the active editor
Human Review	Developer evaluation of generated content before relying on it

---

35. Lesson Summary

GitHub Copilot Chat provides a conversational way to interact with an AI programming assistant directly inside Visual Studio Code.

It can help developers:

- ask programming questions;
- learn concepts;
- request explanations;
- obtain code examples;
- generate functions;
- continue conversations with follow-up questions;
- use development context to produce more relevant answers;
- copy or insert generated code into source files.

One of the most important observations from this lesson is that **the same prompt can produce different results when the surrounding development context changes**.

For example:

"How to get a month from date?"

without clear context may require Copilot to make assumptions.

But:

"How to get a month from date?"

+

Relevant JavaScript context

can help Copilot infer that a JavaScript solution is appropriate.

An even stronger approach combines relevant context with an explicit prompt:

```
Relevant Development Context
            +
Clear Developer Intent
            +
Specific Requirements
            ↓
Better-Defined Request
            ↓
Potentially More Relevant Response
```

Finally, remember that Copilot Chat is an **assistant**, not a substitute for software engineering judgment.

The appropriate workflow remains:

```
Ask
 ↓
Review
 ↓
Understand
 ↓
Refine
 ↓
Insert / Implement
 ↓
Run
 ↓
Test
 ↓
Improve
```

FINAL NOTE

The objective is not simply to make Copilot generate code. The objective is to use Copilot effectively while remaining capable of understanding, evaluating, testing, and maintaining the software that you produce.